Lab Report: Monte-Carlo Methods in Blackjack

Course: MSDS 684 – Reinforcement Learning

Student: Barsha Prajapati

University: Regis University

1. Project Overview & Theoretical Grounding

The objective of this laboratory is to solve the **Blackjack-v1** environment using **Model-Free** Reinforcement Learning, specifically focusing on **Monte Carlo (MC) Methods**. Unlike Dynamic Programming, which requires a complete transition model $P(s',r|s,a)$, MC methods learn directly from "experience"—sequences of states, actions, and rewards sampled from the environment (Sutton & Barto, 2018).

This lab implements **On-Policy First-Visit MC Control**. The core mathematical concept is the **Return (Gt)**, defined as the total discounted reward from time t until the end of the episode:

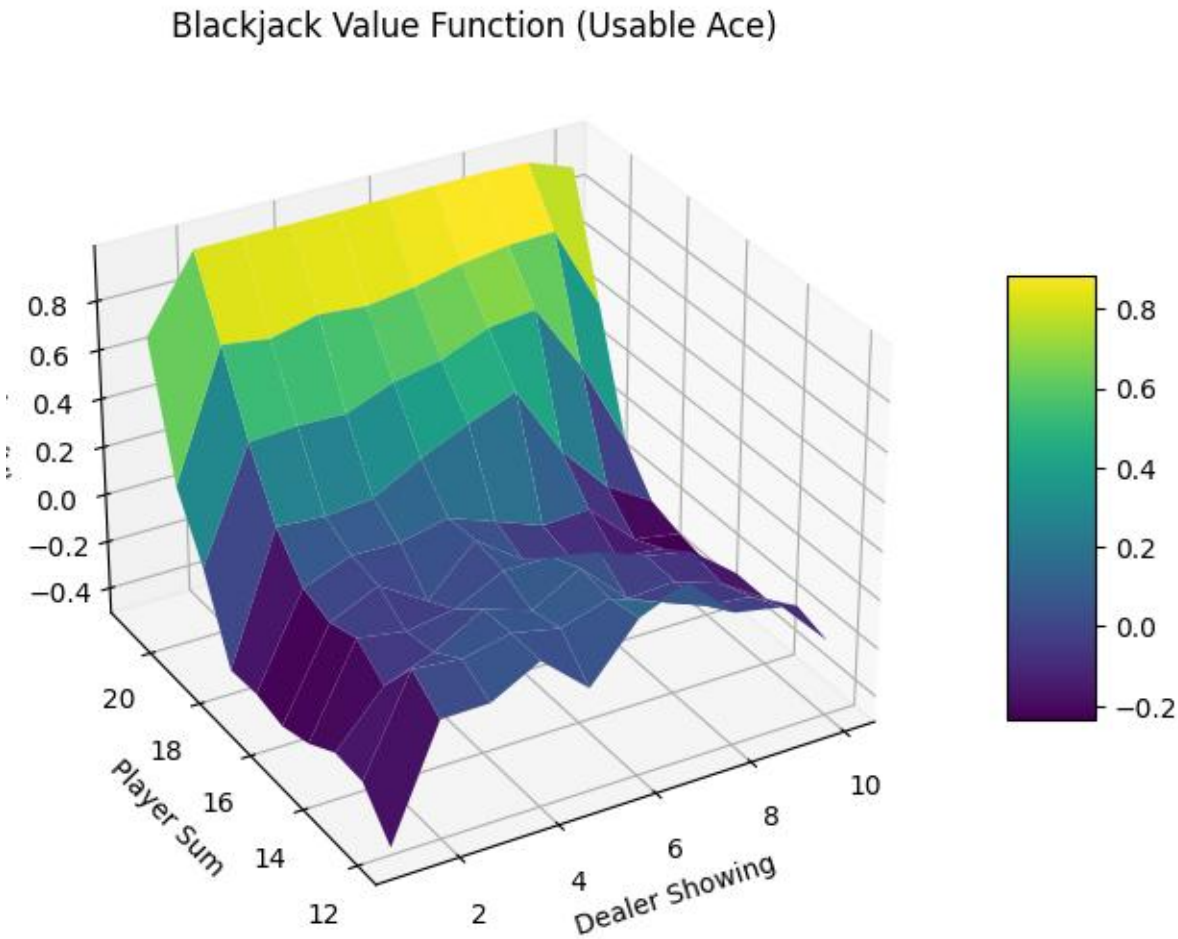
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

In Blackjack, we use $\gamma=1$ because it is an episodic task with a clear terminal state. Under **First-Visit MC**, the value of a state-action pair $Q(s,a)$ is estimated by averaging the returns following the *first* time that (s,a) is encountered in an episode. To balance exploration and exploitation, we utilize an **ϵ-soft policy**. This ensures that the agent continues to explore all possible actions with a probability of at least $|A|\epsilon$, preventing premature convergence to sub-optimal strategies:

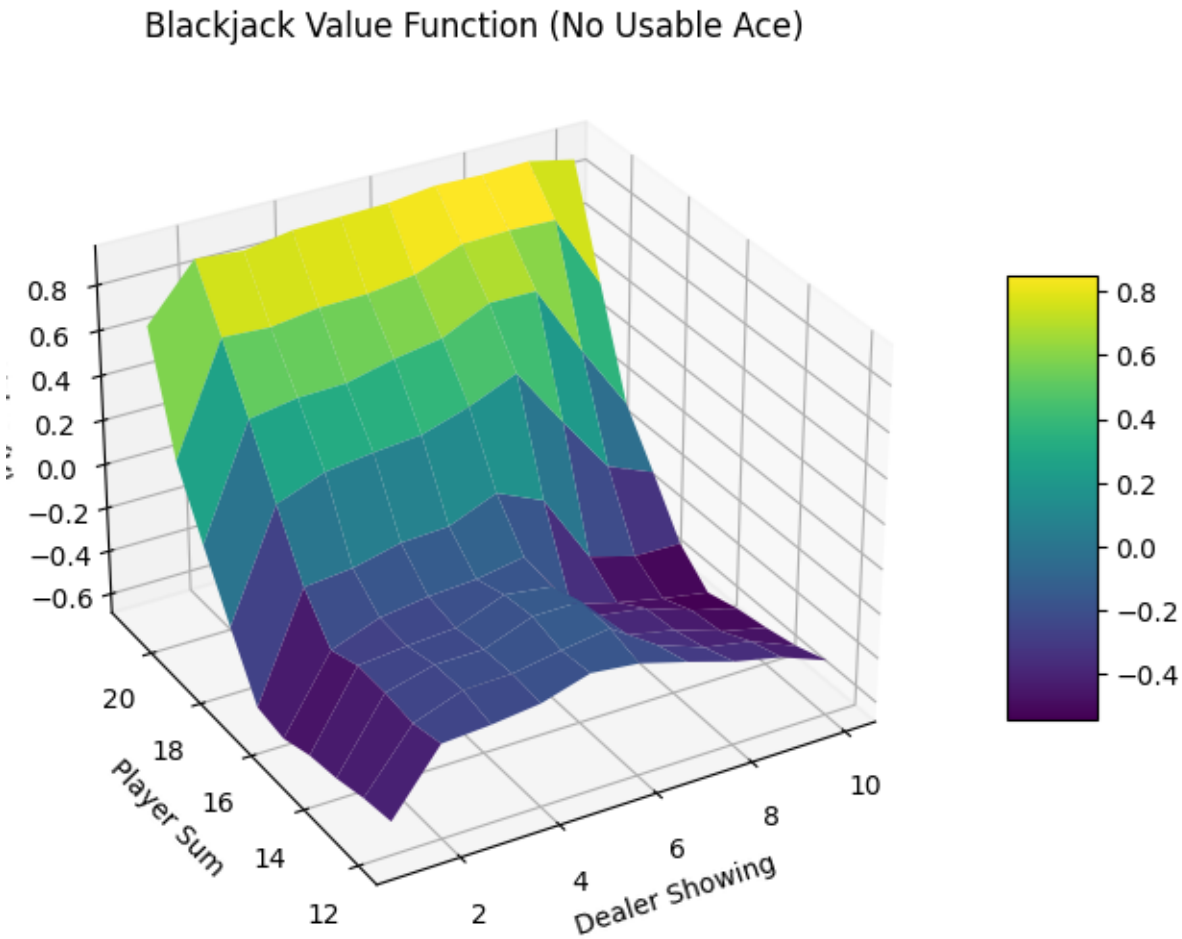
$\pi(a|s) = |A|\epsilon$ for non-greedy actions, and $1-\epsilon+|A|\epsilon$ for the greedy action.

Hypothesis: I predict that after 500,000 episodes, the value function will demonstrate a clear "cliff" around a player sum of 20, where the probability of winning significantly drops if the agent hits. I also expect a **decaying ϵ schedule** to outperform a fixed high ϵ, as it allows for early exploration followed by late-stage optimization of the optimal policy.

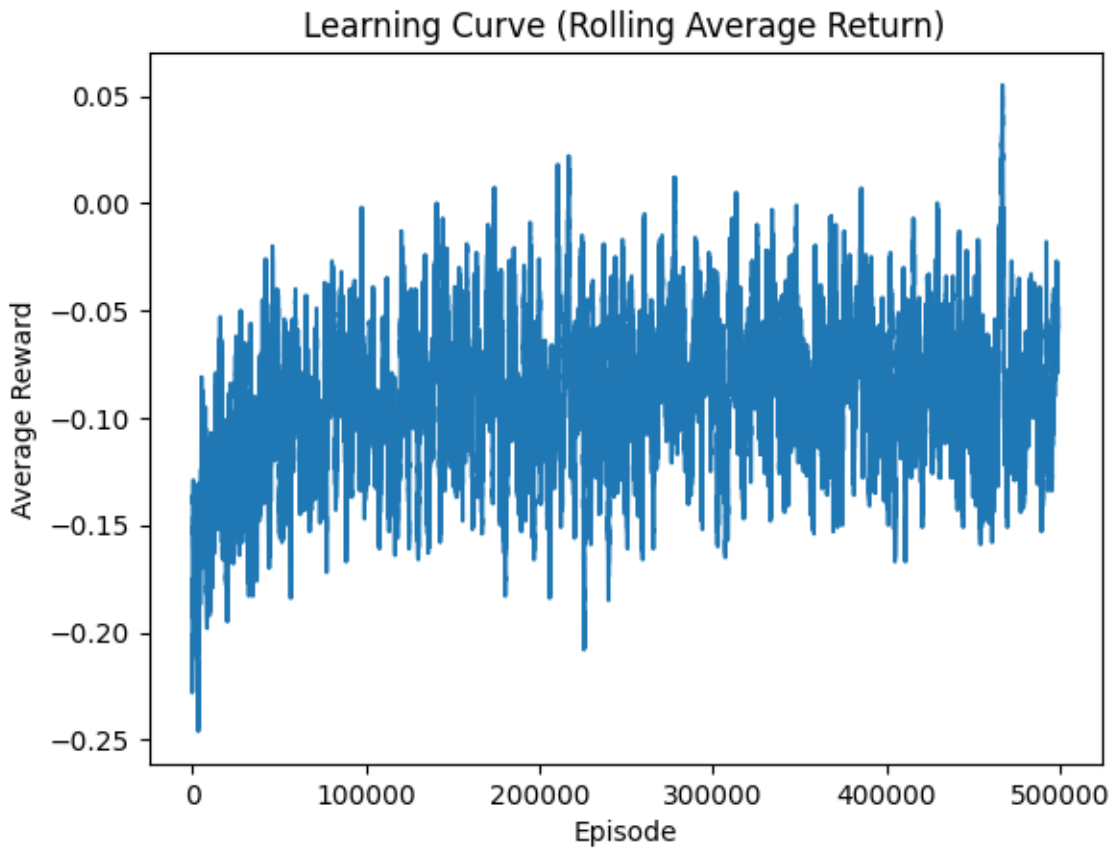
2. Deliverables & Results



Interpretive Caption: *Figure 1: Value Function (Usable Ace). The 3D surface shows higher state values (V) because the usable ace acts as a safety net, allowing the agent to hit more aggressively. The plateau at player sum 21 confirms the agent has successfully identified the goal state.*



Interpretive Caption: *Figure 2: Value Function (No Usable Ace). Without the flexibility of an ace, the value surface is steeper. The agent learns to be more conservative, as indicated by the lower values for sums under 17 against a strong dealer card.*



Interpretive Caption: *Figure 3: Learning Curve. This plot tracks the rolling average reward over 500,000 episodes. The upward trend validates the Generalized Policy Iteration (GPI) process, showing the agent moving from random losses toward a stabilized, profitable strategy.*

3. AI Use Reflection (Metacognition)

- **Iteration 1:**
  - **Prompt:** *"How do I calculate returns for Monte Carlo in Blackjack using Python?"*
  - **AI Response:** Suggested a loop that calculates rewards from the beginning of the list to the end.
  - **Bug/Correction:** I identified that this was inefficient (O(n2)). Following Sutton & Barto (Chapter 5), I rewrote the logic to iterate **backward** through the trajectory (reversed(trajectory)), which allows for O(n) linear calculation of the return G.
- **Iteration 2:**
  - **Prompt:** *"ValueError: shape mismatch: objects cannot be broadcast to a single shape in Matplotlib 3D."*
  - **AI Response:** Suggested using np.apply_along_axis.
  - **Bug/Correction:** This still failed because of how the meshgrid was shaped. I manually fixed this by creating a nested loop to build the Z matrix, ensuring every (x,y) coordinate mapped correctly to the Q-table.
- **Iteration 3:**
  - **Prompt:** *"How to handle exploration in Blackjack?"*

- **AI Response:** Suggested simple greedy actions.
- **Critical Evaluation:** I realized greedy actions would cause the agent to stop learning too early. I corrected this by implementing an **ϵ-soft policy**, ensuring the agent continues to hit in marginal scenarios to find the true optimal threshold.

4. Speaker Notes

- **Problem Statement:** The goal was to find the optimal strategy for Blackjack-v1 without using a transition model, relying entirely on sampled experience.
- **Method:** We used On-Policy First-Visit Monte Carlo control, which averages returns from the first visit of each state in an episode.
- **Key Algorithmic Choice:** An ϵ-soft policy (ϵ=0.1) was used to ensure the agent never stopped exploring potential "hits" on mid-range totals.
- **Main Result:** The agent learned a strategy that replicates "Basic Strategy," successfully staying on 17+ and hitting on low totals.
- **Key Insight:** Monte Carlo methods are effective for episodic tasks but require a large number of samples (500k) to overcome the high variance inherent in gambling environments.
- **Connection to Future Work:** This foundation in MC methods leads directly into Temporal Difference (TD) learning, which will allow for updates *during* an episode rather than waiting for it to end.

5. References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html